University of Waterloo
Faculty of Mathematics

Final Project Report

STAT 441: Statistical Learning - Classification

Prepared by
Darren Lam Kin Teng (20977843)

Contents

Group Information 3
Data Description 3
Models Implemented 3
Methodology 4
Data Preprocessing 4
Model Building 5
Hyperparameter Tuning 6
Result 7
Contributions 9
Appendix 10
Results for each model 10
Bibliography 12

Group Information

Group members:

1. Darren Alexander Lam Kin Teng (20977843)
2. Ojus Udagani (20956639)
3. Raghuv Bansal (21013455)

Data Description

The dataset size is 1.5GB and is a composition of astrophysical images downloaded from Kaggle with 12 target labels namely, Asteroid, Black Hole, Earth, Galaxy, Jupiter, Mars, Mercury, Neptune, Pluto, Saturn, Uranus and Venus. of each image being our target variable. The data folder was already separated into subfolders of training, validation and test folder with each storing 2416, 658 and 345 images respectively. All images are in RGB colour format, and each varied in size. These were later transformed to a uniform pixel size in the data preprocessing stage.

Models Implemented

Given the nature of our dataset, our implementation was limited to using Convolutional Neural Networks (CNN). Consequently, each group member implemented a different CNN. Acknowledging the complexity and scope of developing such models from scratch, which extends beyond the practical focus of this course, we opted to implement 2 pre-trained models, VGG16 and ResNet50, and a custom shallow CNN. As a result, we hypothesized that either ResNet50 or VGG16 would have better model performance.

Methodology

The general pipeline for each implementation is seen in Figure 1:

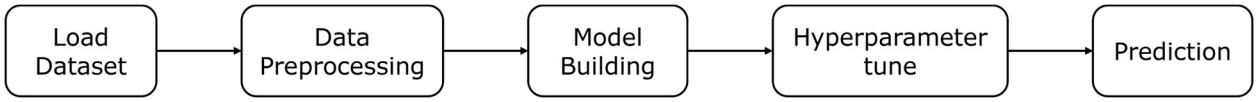

Figure 1 - Model Development Pipeline

Loading Dataset

The dataset was loaded using the Keras function 'image_dataset_from_directory' and specified the training, validation and test batches. The function specified the default batch size 32 to ensure computational efficiency, inferred from our directory structure.

Data Preprocessing

The dataset provided was cleaned and organized in the correct file formats to be used for model implementation. Data preprocessing followed a three-step process.

1. Re-sizing the images

All images were resized to 224×224 pixels and maintained their 3 colour channels (RGB) to ensure a uniform input shape is fed to our models. Therefore, the pixel values $(224 \times 224 \times 3)$ serve as the covariates. Though an arbitrary pixel sized could be used, 224×224 pixels is the default image size used for pre-trained models VGG16 and ResNet50.

2. Data augmentation

Image augmentation was applied to provide random transformations to have new variations of the same image. Techniques included random rotation, random shear, random zoom and random

flipping. Since our training data is relatively small, this step helps to "increase the volume, quantity and quality of our train data" as per Mumuni and Mumuni (2022).

3. Preprocessing

While using pre-trained models, we leverage their preprocessing function on the augmented dataset. The VGG16 preprocessing functions converts the image channels from RGB to BGR and zero centers each color channel according to the means calculated from ImageNet dataset.

Model Building

While developing for multi-class predictions, Sparse Categorical Cross Entropy is used as the loss function – each class is labeled as an integer in the range 0-11. Accuracy would be used as our performance measure for hyperparameter tuning and to compare the three models.

For pre-trained models, VGG16 and ResNet50, transfer learning was employed as the latter models were originally trained on the ImageNet dataset comprising of roughly 14 million terrestrial images. As a result, we do not retrain on their base layers to leverage their pre-trained features. The models were fine tuned to the astrophysical dataset by adding a drop out layer and new fully connected (Dense) layers. The dropout layer would prevent overfitting and set at 0.5 as it "seems to be close to optimal for a wide range of networks and tasks" according to Srivastava et al. (2014).

The custom CNN architecture was developed using an exploratory and heuristic approach rather than a formal and mathematically optimized method. This is due to the complexity of optimally

determining the number of neurons and hidden layers which is a challenge requiring extensive hyperparameter tuning.

Furthermore, all three models implement the Adam optimizer. According to Kingma and Lei Ba (2015) empirical results suggest that "Adam works well in practice and compares favourably to other optimizers."

Hyperparameter Tuning

To ensure a fair comparison between models, each model was tuned over the same set of parameters, namely epochs (e), learning rate (α) and weight decay (λ). The range of values are seen in Table 1.

$\boldsymbol{\alpha}$	0.0001	0.001	0.01
$\boldsymbol{\lambda}$	0.0	0.0001	0.001
$\boldsymbol{e}$	4	8	12

Table 1 - Hyperparameter Values

A grid search approach was used to train the models over every combination of (λ, α) and only over 12 epochs. Therefore, at each checkpoint epoch, training accuracy, validation accuracy and loss metrics were recorded for the model. This effectively reduces the number of runs required from 27 to 9 runs.

While the default learning rate in Keras and PyTorch is set at 0.001, 2 arbitrary learning rates were chosen such that they were an order of magnitude apart. Weight decay has a default value of 0,

suggesting no regularization, and 2 arbitrary values an order of magnitude apart. These values would be used in combination with L2 regularization.

Results

The results for each model are displayed in the appendix is displayed in figures 2,3 and 4 of the appendix. From our implementations, ResNet50 outperformed the others with parameters $(\lambda, \alpha) = (0, 0.001)$, over 12 epochs on the training and validation sets. The model had a validation accuracy of 98.78% and a prediction accuracy of 96.22%. Additionally, VGG16's performance was close with a validation accuracy of 97.26% and prediction accuracy of 94.78% over the same parameters. While the custom CNN performed best on parameters $(\lambda, \alpha) = (0.0001, 0.001)$ on 12 epochs with a validation accuracy 92.40% and prediction accuracy 93.04%.

	ResNet50	**VGG16**	**Custom CNN**
Validation Accuracy	98.78%	97.26%	92.40%
Prediction Accuracy	96.22%	94.78%	93.04%

Table 2 - Overall Model Performance

Comparing for the same epochs and learning rate, we find that weight decay had minimal to no impact on the overall performance of the models. However, we find that usually for a weight decay 0.0001, the validation accuracy was typically lower than for 0 weight decay. This may suggest that may be slightly too high for our specific dataset and may cause potential underfit.

Furthermore, comparing for the same epoch and weight decay, learning rate had the most impact on model performance. For every combination of epoch and learning rate, the accuracy

significantly improved as we move the learning rate to 0.001 – confirming that it was a suitable value for our training. In comparison, a learning rate of 0.01 had the worst performance suggesting that its step size was too high and overshooting the minimum to converge.

Due to the complexity of deep CNNs, namely VGG16 and ResNet50, the models can offer high predictive performance however, the trade-off is their interpretability. A complete human interpretation of the feature weights is not feasible now. However, new advancements in the field of explainable AI (XAI) could provide a proxy to help with model interpretation.

As mentioned previously, model development and hyper-parameter tuning of models have been largely limited due to compute power available. With more resources, available this project could be improved by tuning over a larger range of values for weight decay, learning rate and epochs to maximize the model performance.

Beyond hyperparameter tuning, our focus could be on data augmentation by further applying geometric transformations, kernel filters, color space transformation and random erasing on the available images. Data augmentation would help the model better generalize unseen data.

Contributions

The project contributions were split evenly among the three members. Each model implementation was undergone by one group member. The table below displays the model and the member that implemented it.

Model	**Group Member**
VGG16	Darren Lam Kin Teng
Custom CNN	Ojus Udgani
ResNet50	Raghuv Bansal

Appendix

GitHub project submission

https://github.com/dlkt101101/STAT441-Project

Kaggle Dataset

https://www.kaggle.com/code/a3amat02/astrophysical-objects-classification-using-resnet#ResNet-fine-tuning-process-and-training-logs

Results for each model

Learning Rate	0.01	0.01	0.01	0.001	0.001	0.001	0.0001	0.0001	0.0001
Weight Decay	0	0.001	0.0001	0	0.001	0.0001	0	0.001	0.0001
Epoch									
1	90.5344%	94.5038%	94.8092%	95.8779%	94.1985%	94.0458%	85.3435%	86.7176%	81.5267%
2	94.5038%	96.3359%	97.2519%	95.8779%	96.1832%	96.3359%	93.4351%	92.2137%	93.4351%
3	97.4046%	96.3359%	98.0153%	96.9466%	97.4046%	97.5573%	94.1985%	94.9618%	94.9618%
4	97.4046%	96.3359%	98.0153%	97.0992%	97.8626%	98.1679%	95.4198%	95.8779%	95.7252%
5	97.7099%	96.3359%	98.0153%	97.8626%	98.0153%	98.1679%	95.8779%	96.4885%	96.3359%
6	98.1679%	96.3359%	98.0153%	98.0153%	98.0153%	98.1679%	96.7939%	96.9466%	96.6412%
7	98.1679%	96.3359%	98.0153%	98.6260%	98.1679%	98.1679%	96.9466%	97.0992%	96.9466%
8	98.1679%	96.3359%	98.0153%	98.6260%	98.4733%	98.1679%	97.5573%	97.7099%	97.2519%
9	98.1679%	96.3359%	98.0153%	98.7786%	98.4733%	98.1679%	97.5573%	97.7099%	97.5573%
10	98.1679%	96.3359%	98.0153%	98.7786%	98.4733%	98.1679%	97.5573%	97.7099%	97.5573%
11	98.1679%	96.3359%	98.0153%	98.7786%	98.4733%	98.1679%	97.5573%	97.7099%	97.7099%
12	98.1679%	96.3359%	98.0153%	98.7786%	98.4733%	98.1679%	97.5573%	97.7099%	97.8626%

Figure 2 - ResNet50 Results

accuracy	loss	val_accuracy	val_loss	epoch	weight_decay	learning_rate
0.895695	0.37538	0.972644	0.118234	12	0	0.001
0.840232	0.701719	0.962006	0.146486	4	0	0.001
0.878311	0.525721	0.960486	0.156278	12	0	0.0001
0.851821	1.241861	0.960486	0.660452	8	0.001	0.0001
0.889073	0.490573	0.958967	0.231452	12	0.0001	0.001
0.869619	0.63347	0.957447	0.230503	12	0.0001	0.0001
0.875414	1.060976	0.955927	0.782487	8	0.001	0.001
0.850166	0.746546	0.954407	0.171734	8	0	0.0001
0.854305	0.836131	0.954407	0.235968	8	0.0001	0.0001
0.878725	0.559852	0.954407	0.30541	8	0.0001	0.001
0.871689	1.092979	0.951368	0.668903	12	0.001	0.0001
0.791805	1.197263	0.948328	0.222743	4	0	0.0001
0.876656	0.43226	0.948328	0.234895	8	0	0.001
0.831954	0.816753	0.943769	0.255857	4	0.0001	0.001
0.898593	0.964029	0.940729	0.831993	12	0.001	0.001
0.802152	1.296083	0.93769	0.398562	4	0.0001	0.0001
0.807119	1.767249	0.93465	0.785975	4	0.001	0.0001
0.835265	1.264954	0.913374	0.903823	4	0.001	0.001
0.68957	2.785592	0.870821	2.033765	12	0.0001	0.01
0.636176	1.571782	0.817629	0.819062	4	0	0.01
0.685844	1.28361	0.81003	0.73302	8	0	0.01
0.668046	2.845886	0.803951	2.286359	8	0.0001	0.01
0.595199	8.860792	0.794833	7.926706	8	0.001	0.01
0.634934	7.154312	0.778116	6.766174	12	0.001	0.01
0.578228	10.97522	0.758359	10.016748	4	0.001	0.01
0.660596	1.306526	0.753799	0.812447	12	0	0.01
0.611341	2.923431	0.670213	2.610455	4	0.0001	0.01

Figure 3 - VGG16 Results

Learning Rate	Weight Decay	Epoch	Training Accuracy	Validation Accuracy
0.0001	0.0000	4	75.165563	79.787234
0.0001	0.0000	8	81.746689	82.978723
0.0001	0.0000	12	85.430464	85.410334
0.0001	0.0010	4	76.862583	79.635258
0.0001	0.0010	8	81.995033	84.194529
0.0001	0.0010	12	84.726821	84.650456
0.0001	0.0001	4	76.324503	80.395137
0.0001	0.0001	8	81.746689	85.866261
0.0001	0.0001	12	84.395695	87.841945
0.0010	0.0000	4	81.456954	84.650456
0.0010	0.0000	8	88.038079	87.993921
0.0010	0.0000	12	90.066225	88.449848
0.0010	0.0010	4	80.091060	81.306991
0.0010	0.0010	8	87.541391	88.449848
0.0010	0.0010	12	88.617550	88.753799
0.0010	0.0001	4	81.953642	86.170213
0.0010	0.0001	8	86.879139	87.082067
0.0010	0.0001	12	90.107616	89.209726
0.0100	0.0000	4	34.892384	40.121581
0.0100	0.0000	8	45.985099	43.009119
0.0100	0.0000	12	49.875828	53.039514
0.0100	0.0010	4	8.899007	9.270517
0.0100	0.0010	8	8.774834	9.270517
0.0100	0.0010	12	8.650662	6.382979
0.0100	0.0001	4	54.014901	62.917933
0.0100	0.0001	8	61.216887	67.933131
0.0100	0.0001	12	35.554636	37.993921

Figure 4 - Custom CNN Results

Bibliography

Mumuni, A. and Mumuni, F. (2022). Data augmentation: A comprehensive survey of modern approaches. *Array*, 16, p.100258. doi:https://doi.org/10.1016/j.array.2022.100258.

Keras (n.d.). *Keras documentation: VGG16 and VGG19*. [online] keras.io. Available at: https://keras.io/api/applications/vgg/.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Journal of Machine Learning Research, [online] 15(56), p.2. Available at: https://jmlr.org/papers/v15/srivastava14a.html [Accessed 3 Dec. 2025].

Kingma, D. and Lei Ba, J. (2015). *ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION*. [online] *arXiv*, p.1. Available at: https://arxiv.org/pdf/1412.6980 [Accessed 3 Dec. 2025].